



Факултет по Математика и Информатика
ПУ "Паисий Хилендарски"

JWT, TOKEN-BASED AUTHENTICATION И SINGLE SIGN-ON (SSO)

Витали Атиас

Проблемът със session-based authentication в съвременните системи

В предишната лекция разгледахме session-based authentication, при който сървърът създава session ID и го изпраща на клиента чрез cookie. Сървърът съхранява информацията за сесията и при всяка заявка проверява session ID.

Този подход работи добре за традиционни уеб приложения, но има проблеми в съвременни архитектури като:

- Single Page Applications (SPA)
- Mobile приложения
- Microservices архитектури
- Distributed systems

Проблемът със session-based authentication в съвременните системи

Основният проблем е, че session-based authentication изисква сървърът да съхранява информация за всяка сесия.

Пример:

SESSIONID=abc123 → user=student1

SESSIONID=xyz456 → user=student2

Ако системата има милиони потребители, сървърът трябва да съхранява милиони сесии. Освен това, ако има множество сървъри (load balancing), всички сървъри трябва да имат достъп до session информацията.

Token-based authentication – основна идея

При token-based authentication, вместо сървърът да съхранява информация за сесията, клиентът получава token, който съдържа информация за потребителя.

Token е криптографски подписан низ, който удостоверява идентичността на потребителя.

Пример token:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Token-based authentication – основна идея

Клиентът изпраща този token при всяка заявка:

GET /profile

Authorization: Bearer

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Сървърът проверява token, без да съхранява session.

Този подход се нарича stateless authentication.

Stateless означава, че сървърът не съхранява състояние.

JWT – JSON Web Token

JWT е стандартен формат за token-based authentication.

JWT е дефиниран в RFC 7519.

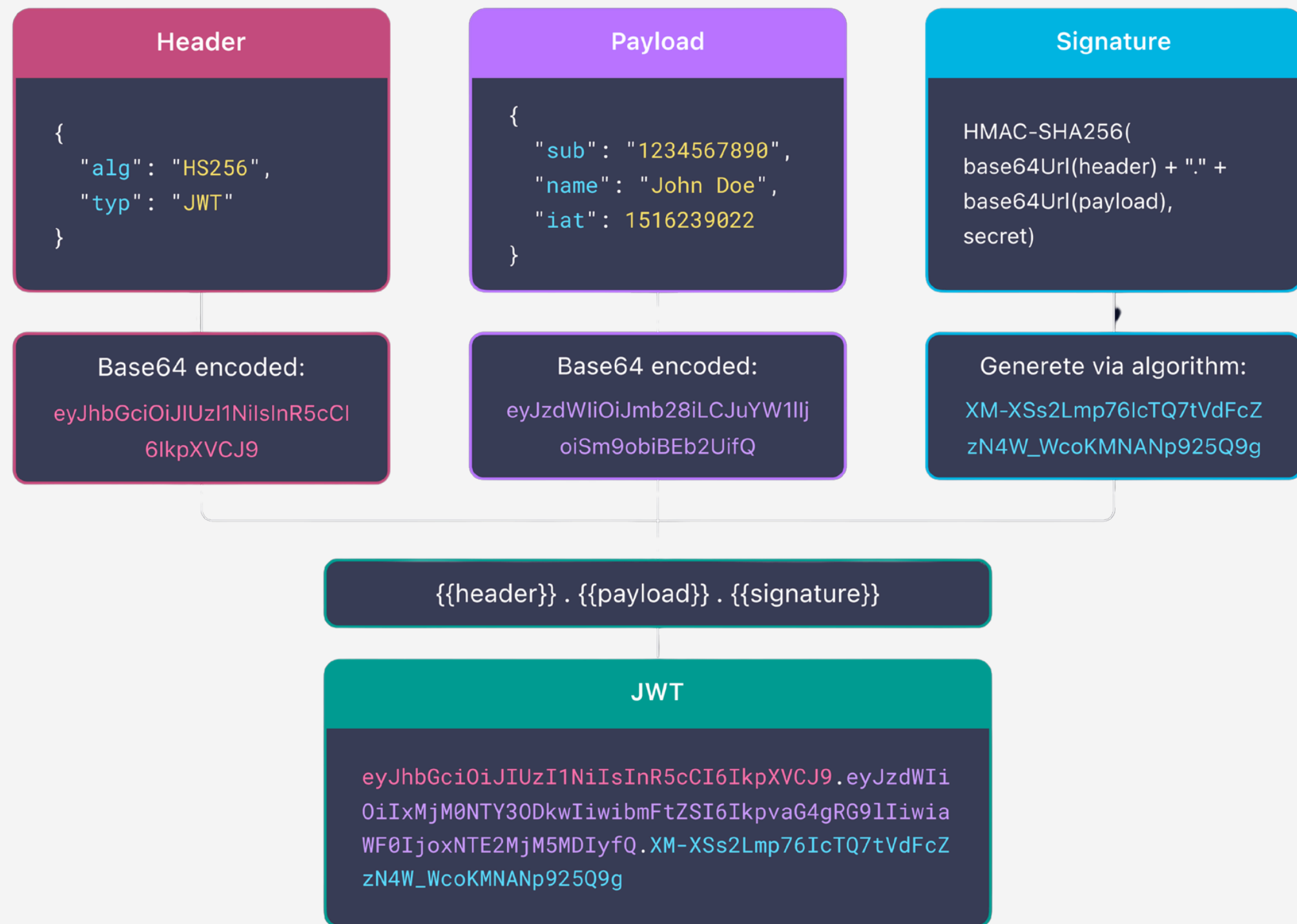
JWT е текстов низ, който съдържа три части:

header.payload.signature

Пример:

eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyljojYWRTaW4ifQ.abc123signature

JWT – JSON Web Token



JWT – JSON Web Token

Header

Съдържа информация за алгоритъма.

Поleta:

- alg – алгоритъм за подписване
- typ – тип (JWT)

Често използвани алгоритми:

- HS256 (HMAC SHA-256)
- RS256 (RSA)

JWT – JSON Web Token

Стандартни claims:

- sub – subject (идентификатор)
- exp – expiration time
- iat – issued at
- iss – issuer
- aud – audience

Payload

Payload съдържа claims. Payload не е криптиран – той е Base64 encoded. Не трябва да съдържа, чувствителни лични данни

Пример:

```
{  
  "sub": "1234567890",  
  "username": "student1",  
  "role": "student",  
  "exp": 1712345678,  
  "iat": 1712342000  
}
```

JWT – JSON Web Token

Signature

Signature се използва за защита на token. Ако payload бъде променен, signature ще бъде невалиден.

Signature се създава чрез:

signature = hash(header + payload + secret)

HMACSHA256(

base64UrlEncode(header) + "." +

base64UrlEncode(payload),

secret*

)

***secret е тайна стойност, известна само на сървъра.**

JWT vs Session-based Authentication

Критерий	Sessions	JWT
Състояние	Stateful	Stateless
Сървърна памет	Изисква	Не изисква
Scaling	По-сложно	По-лесно
Logout	Лесно	Трудно
Token revocation	Лесно	Трудно

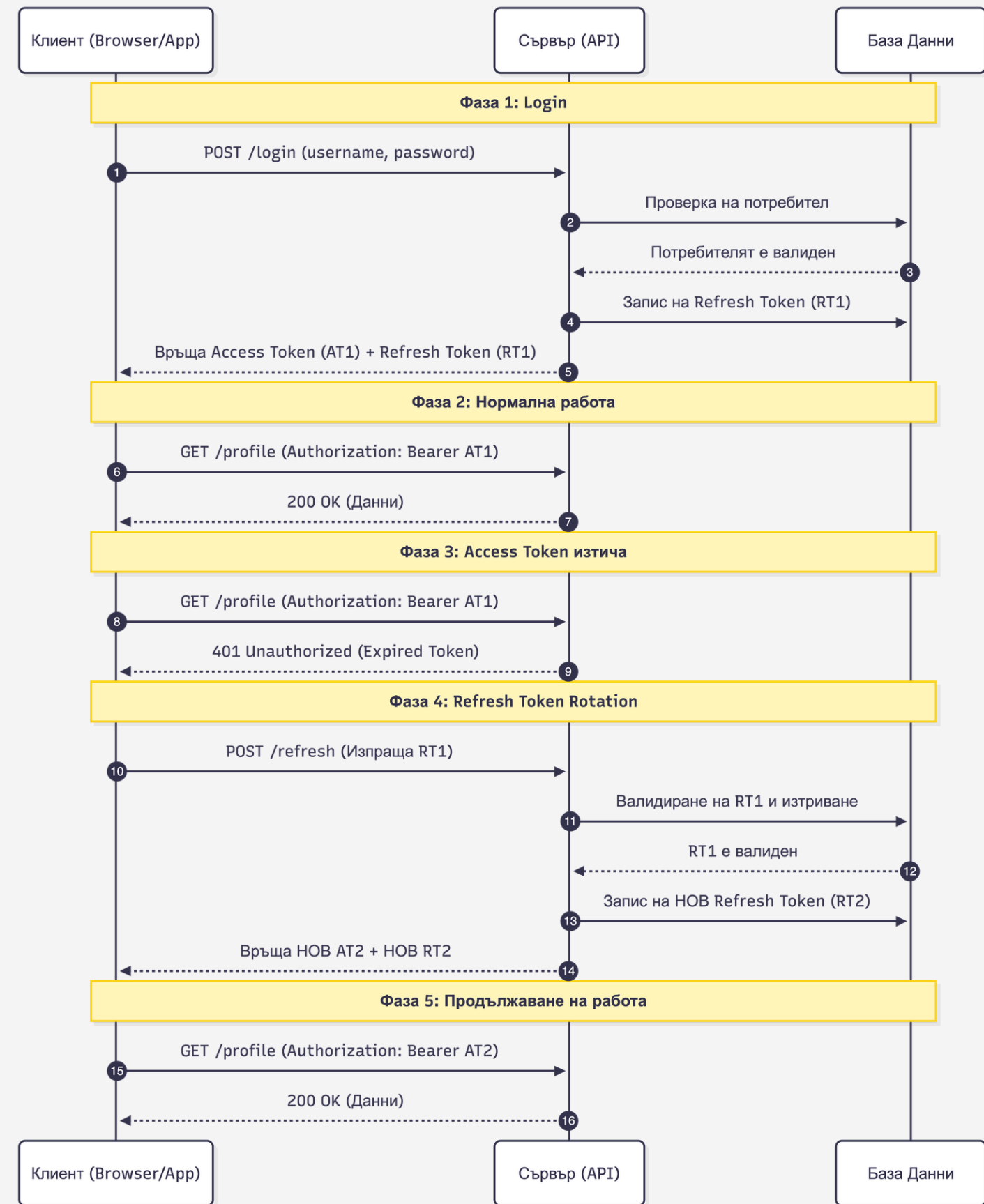
Как работи JWT authentication – пълен процес

- **Login заявка** - клиентът изпраща потребителско име и парола към /login.
- **Генериране на двойка токени** - сървърът проверява данните и генерира:
 - Access Token (AT) - краткотраен (напр. 15 мин), за ежедневни заявки.
 - Refresh Token (RT) - дълготраен (напр. 7 дни), за получаване на нов AT.
- **Запис в базата данни** - сървърът съхранява RT в базата данни (свързан с потребителя), за да може да го инвалидира при нужда.
- **Изпращане на клиента** - клиентът получава и двата токена (препоръчително е RT да е в HttpOnly бисквитка).

Как работи JWT authentication – пълен процес

- **Редовна работа** - клиентът използва AT за достъп до защитени ресурси (GET /profile).
- **Изтичане на AT** - когато AT изтече, сървърът връща 401 Unauthorized.
- **Заявка за опресняване (Rotation)** - клиентът изпраща своя RT към /refresh.
- **Валидация и Ротация** - сървърът проверява RT в базата данни. Ако е валиден:
 - Анулира стария RT.
 - Генерира чисто нов AT и чисто нов RT.
 - Обновява базата данни с новия RT.
- **Нова двойка токен** - Клиентът съхранява новите токени и продължава работа.

Как работи JWT authentication – пълен процес



Недостатъци на JWT

JWT има и недостатъци:

- **Token theft** - Използвай HttpOnly cookies, за да спреш кражбата чрез XSS скриптове.
- **Logout** - Тъй като JWT е stateless, решението е Blacklisting (черен списък) в Redis, където се пазят невалидните токени до техния ехр.
- **Expiration** - Използвай кратък живот на Access Token (напр. 15 мин.) в комбинация с Refresh Token, за да намалиш прозореца за злоупотреба.

JWT може да се съхранява:

- в memory
- в cookie

Съхраняването в cookie е по-сигурно, ако cookie има HttpOnly flag.

JWT amaku

- **Tampering** - защитата тук е само една – никога не се доверявай на payload-а, без да си проверил подписа.
Сървърът трябва да използва библиотеки за верификация, които автоматично отхвърлят токена, ако дори един бит е променен.
- **XSS Theft** - ако държим JWT в localStorage, сме уязвими.
Решението е HttpOnly флагът за бисквитки, който буквално "забранява" на JavaScript (и съответно на атакуващия) да чете токена.

SSO (Single Sign-On)

SSO (Single Sign-On) е метод за аутентикация, който позволява на потребителя да достъпва множество независими системи с едни и същи входни данни след еднократно вписване.

Основни характеристики:

- **Централизация** - един Identity Provider (IdP) управлява потребителските профили и пароли.
- **Удобство (UX)** - спестява време и предотвратява „умората от пароли“ (password fatigue).
- **Сигурност** - намалява повърхността за атака, тъй като паролата се въвежда само на едно (защитено) място.
- **Протоколи** - Базира се на стандарти като OpenID Connect (OIDC) или SAML.

Identity Provider (IdP)

Identity Provider (IdP) е централният authentication сървър (Google, Microsoft, Okta). Неговата роля е да потвърди самоличността на потребителя веднъж .

Основни отговорности на IdP:

1. **Аутентикация (Authentication)** - директно проверява кой си ти (парола, биометрия, MFA).
2. **Управление на сесии (Session Management)** - поддържа централна сесия (обикновено чрез httpOnly бисквитка),
3. **Издаване на токени** - след успешен вход, IdP генерира подписани JWT (ID Tokens) или SAML асерции, които съдържат данни за потребителя (Claims).
4. **Справочник на потребители** - съхранява или се свързва с база данни (напр. Active Directory или LDAP).

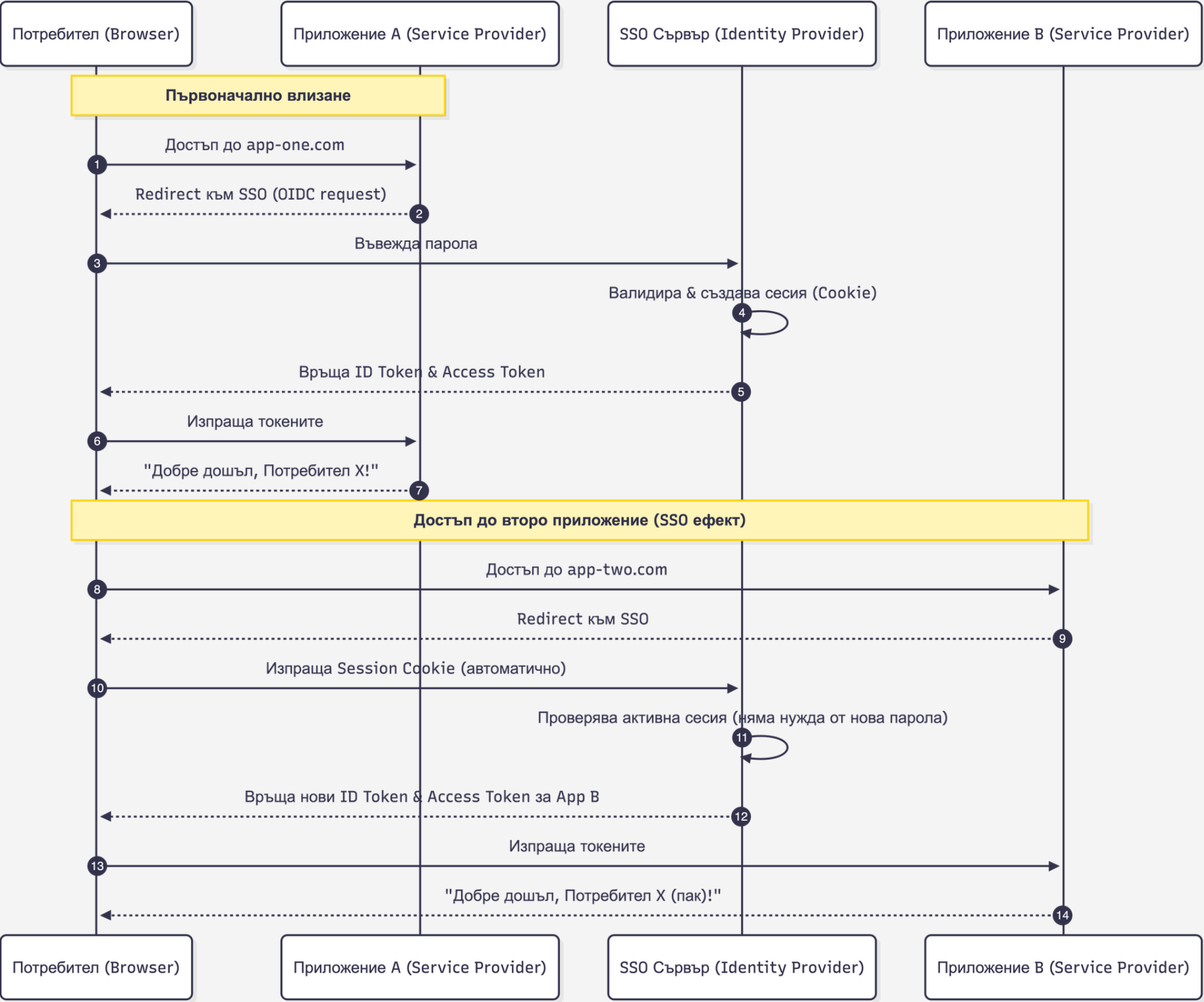
OAuth 2.0 и OpenID Connect

- **OAuth 2.0** – authorization framework
- **OpenID Connect** – authentication layer върху OAuth 2.0

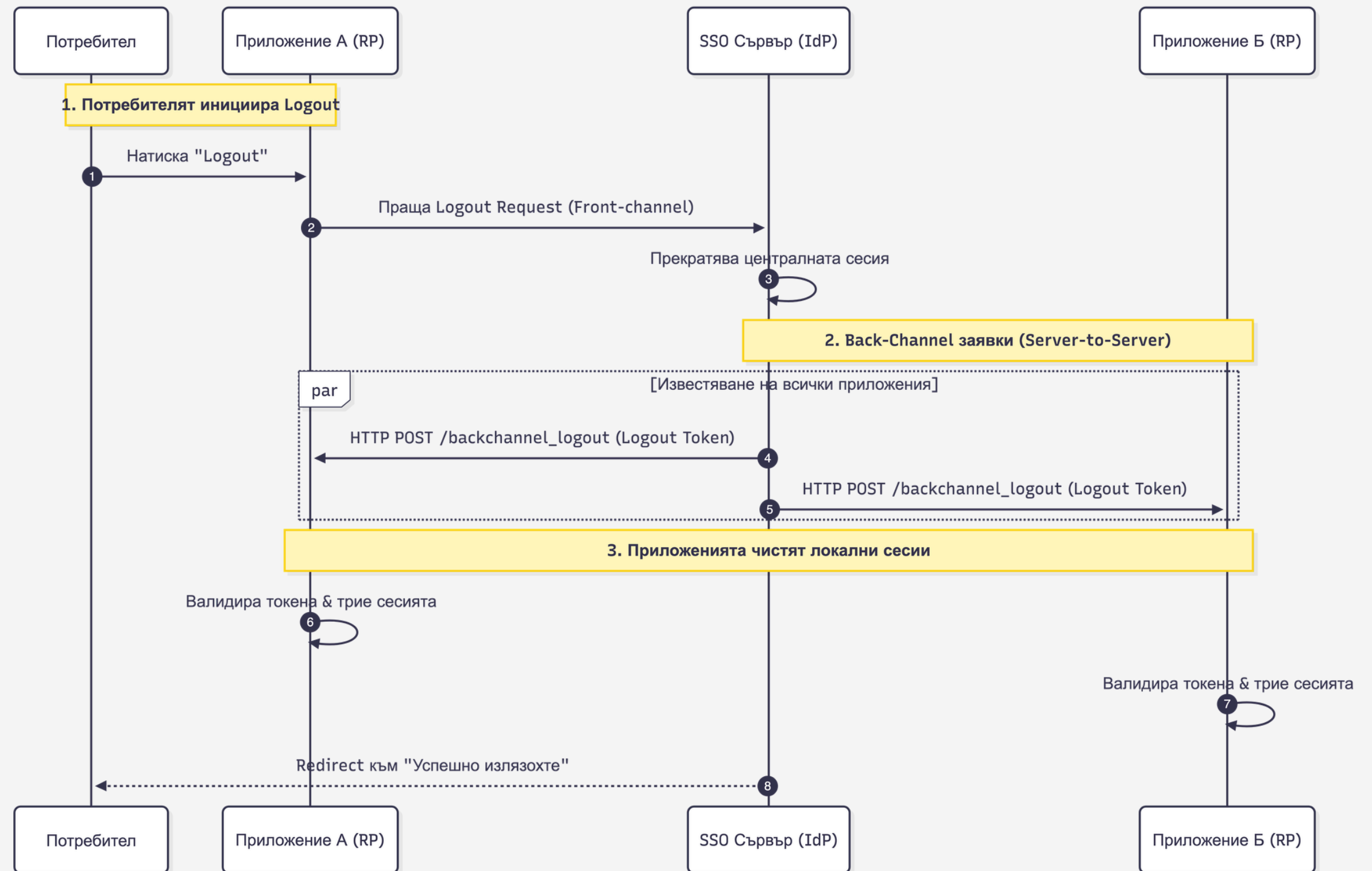
OAuth е за достъп (разрешение), докато OpenID Connect е за идентификация (кой си ти). Повечето модерни системи използват и двете едновременно.

Технология	Какво представлява?	Аналогия
OAuth	Протокол за достъп	Ключ-карта за хотел (отваря врати).
OIDC	Протокол за идентичност	Снимката и името върху картата.
SSO	Потребителско изживяване	Една карта за стаята, фитнеса и спа центъра.

SSO (Single Sign-On)



SLO (Single Log-Out)



Trust models и рискове при SSO

Single Point of Failure (IdP)

SSO централизира доверието в Identity Provider (IdP).

Компрометиране на IdP = издаване на валидни токени към всички свързани системи.

Риск:

- Масов неоторизиран достъп
- Компрометирани signing keys = пълен контрол

Token Forwarding (Privilege Escalation)

Ако Приложение Б приеме токен, предназначен за Приложение А е възможна ескалация на достъп. Затова проверката на aud (Audience) е задължителна.

Trust models и пускове при SSO

Задължителни проверки във всеки Service Provider (SP)

Всеки SP сам валидира токена:

- Signature → защита от tampering
- Issuer (iss) → токенът е от легитимния IdP
- Audience (aud) → токенът е издаден за точно това приложение
- Expiration (exp) → токенът не е изтекъл